

Technical Audit Report - Recur Standard

Executive Summary

Audit Date : November 10, 2025

Project : Recur Standard (RIP-001 to RIP-008)

Version Analyzed : v1.0.0

Project Type : Permissioned-pull standard for cross-domain value movements

13

Issues Identified

2

Critical

4

High

5

Medium

2

Low

Technology Stack

- ▶ **Smart Contracts** : Solidity ^0.8.20
- ▶ **SDK** : JavaScript (ESM) with ethers.js
- ▶ **Standards** : EIP-712, EIP-1271, ERC-20
- ▶ **Architecture** : Non-custodial, permissioned-pull
- ▶ **License** : Apache-2.0

Positive Points

- **Non-custodial architecture** : No contract holds funds, all transfers are direct via `transferFrom()`
- **Replay protection** : EIP-712 with domain separator including `chainId` and contract address
- **Low-s enforcement** : Protection against ECDSA signature malleability
- **Revocation mechanisms** : Multiple levels of kill switches (`registry`, `intents`, `channels`)
- **Documentation** : Complete documentation with RIP-001 to RIP-008
- **Reentrancy guards** : Present on `FlowChannelHardened` and `PolicyEnforcer`
- **Access control** : Well-defined access controls with modifiers (`onlyController`, `onlyGrantor`, etc.)

Identified Issues

CRITICAL #1 : Race Condition on ownerOfAuth in RecurConsentRegistry **CRITICAL**

 File: `contracts/RecurConsentRegistry.sol`, Lines 195-198

A malicious `trustedExecutor` can record a pull with an `authHash` before the real grantor, thus stealing revocation rights.

```
function recordPull(...) external onlyTrustedExecutor { if
(ownerOfAuth[authHash] == address(0)) {
ownerOfAuth[authHash] = grantor; // ⚠ First come, first
served } // ... }
```

Impact : An attacker controlling a `trustedExecutor` can usurp revocation rights of a legitimate authorization, preventing the real grantor from revoking their consent.

Recommendation : Verify that `msg.sender` (or an explicit parameter) matches the expected grantor, or use a cryptographic proof mechanism to bind `authHash` → grantor before the first `recordPull`.

CRITICAL #2 : SDK/Contract Inconsistency - FlowIntent Missing validAfter and metadataHash **CRITICAL**

 File: `sdk/flowIntent.js`, Lines 49-68 vs `contracts/FlowIntentRegistry.sol`

The JavaScript SDK does not sign `validAfter` and `metadataHash` while the Solidity contract expects them in the `FlowIntent` struct.

```
// SDK (flowIntent.js) - MISSING validAfter and
```

```
metadataHash export function buildFlowIntent({ grantor,
executor, token, sourceDomain, destDomain, maxAmount,
validBefore, nonce // ⚠ No validAfter or metadataHash })
{ ... } // Contract (FlowIntentRegistry.sol) - EXPECTS
these fields struct FlowIntent { // ... uint256
validAfter; // ⚠ Missing in SDK uint256 validBefore;
bytes32 metadataHash; // ⚠ Missing in SDK // ... }
```

Impact : Signatures generated by the SDK will not match what the contract expects, causing verification failures and making the SDK unusable.

Recommendation : Add validAfter and metadataHash to the SDK and update FLOW_INTENT_TYPES to include these fields in the EIP-712 signature.

HIGH #1 : Missing SafeERC20 for Non-Standard Tokens **HIGH**

📌 All contracts using transferFrom()

All contracts use IERC20.transferFrom() directly without SafeERC20 wrapper, which can cause issues with non-standard tokens like USDT that don't return a bool.

```
// Current problem
require(IERC20(auth.token).transferFrom(auth.grantor,
auth.grantee, amount), "TRANSFER_FAIL"); // Recommended
SafeERC20.safeTransferFrom(IERC20(auth.token),
auth.grantor, auth.grantee, amount);
```

Impact : Pulls will fail silently with certain tokens (USDT on some chains), causing poor user experience and potentially locked funds.

Recommendation : Use OpenZeppelin's SafeERC20 for all `transferFrom()` calls. This is also mentioned in the informal audit (AUDIT_SUMMARY.md).

HIGH #2 : Overflow Risk in `FlowChannelHardened._sync()` HIGH

 File: `contracts/FlowChannelHardened.sol`, Line 225

The calculation `dt * c.ratePerSecond` can overflow if values are very large, even with Solidity 0.8.20 which has default checks.

```
uint256 dt = block.timestamp - c.lastUpdate; uint256
newAccrued = c.accrued + (dt * c.ratePerSecond); // 
Possible overflow
```

Example : If `dt = 232` (approximately 136 years) and `ratePerSecond = 1018`, the product exceeds `type(uint256).max`.

Impact : If a channel is not used for a very long time with a high `ratePerSecond`, the accrual calculation can overflow and cause a revert, blocking the channel.

Recommendation : Add explicit checks or use a secure math library. Verify that `dt * ratePerSecond <= type(uint256).max - accrued` before the calculation.

HIGH #3 : No Verification that `sourcePullContract` is a Valid Contract HIGH

 File: `contracts/CrossNetworkRebalancer.sol`, Line 270

The contract only checks that `sourcePullContract != address(0)` but does not verify that it is a valid contract with code.

```
require(sourcePullContract !== address(0),  
"BAD_SOURCE_ADAPTER"); // ⚠️ No verification that it's a  
contract ok =  
IRecurPullLike(sourcePullContract).pull(...);
```

Impact : If an EOA address is passed by mistake, the call will fail silently or gasless, but this can cause confusion and debugging issues.

Recommendation : Add

```
require(sourcePullContract.code.length > 0, "NOT_CONTRACT")  
before the call.
```

HIGH #4 : Insecurely Generated Nonce in SDK

HIGH

📌 File: sdk/flowIntent.js, Lines 20-24

The SDK uses `Date.now()` + `Math.random()` to generate nonces, which is not cryptographically secure.

```
function ensureBytes32Nonce(nonce) { if (nonce) return  
nonce; const seed = `${Date.now()}::${Math.random()}`; //  
⚠️ Not secure return keccak256(toUtf8Bytes(seed)); }
```

Impact : Risk of nonce collision if multiple users generate authorizations simultaneously, potentially causing replays or conflicts.

Recommendation : Use `crypto.getRandomValues()` (Web Crypto API) or a cryptographic library to generate secure random nonces.

MEDIUM #1 : Missing Reentrancy Protection on Several Critical Contracts MEDIUM

 `RecurPullSafeV2.sol`, `CrossNetworkRebalancer.sol`,
`AdaptiveRouter.sol`, `SettlementMesh.sol`

Several contracts make external calls without explicit reentrancy protection, relying on trust in the called contracts.

Contract	External Call	Protection
RecurPullSafeV2	<code>registry.recordPull()</code>	✗ None (trusts the registry)
CrossNetworkRebalancer	<code>sourcePullContract.pull()</code>	✗ None
AdaptiveRouter	<code>flow.pull()</code>	✗ None
SettlementMesh	<code>router.routeStep()</code>	✗ None

Impact : If a malicious or compromised contract is passed as a parameter, reentrancy attacks are possible. However, the risk is mitigated as these contracts are expected to be deployed with trusted addresses only.

Recommendation : Add reentrancy guards (`nonReentrant` modifier) on critical functions, or explicitly document that these contracts must be deployed with trusted addresses only.

MEDIUM #2 : EIP-1271 Error Handling Could Be Improved MEDIUM

 File: `contracts/RecurPullSafeV2.sol`, Lines 310-316

The try-catch for EIP-1271 simply returns false on error, which can mask issues.

```
if (auth.grantor.code.length != 0) { try
  IEIP1271(auth.grantor).isValidSignature(digest, signature)
  returns (bytes4 magicVal) { return (magicVal ==
  0x1626ba7e); } catch { return false; // ⚠ Masks the real
  error } }
```

Impact : If an EIP-1271 wallet contract has a bug or reverts for a reason other than "invalid signature", the error is silently ignored, making debugging difficult.

Recommendation : Emit an event on EIP-1271 errors to facilitate debugging, or differentiate between invalid signature errors vs other errors.

MEDIUM #3 : No Verification that controller is a Contract in FlowIntentRegistry **MEDIUM**

📌 File: contracts/FlowIntentRegistry.sol

If controller is an EOA instead of a contract (Safe/multisig), calls could fail unexpectedly.

Impact : If an EOA is configured as controller by mistake, calls to `verifyAndConsume()` will fail, potentially blocking the system.

Recommendation : Add a verification in the constructor or `setController()` to ensure that controller has code (is a contract).

MEDIUM #4 : Naming Inconsistency in SDK

flowIntent.js MEDIUM

 File: sdk/flowIntent.js, Lines 49-68

The SDK uses `sourceDomain` and `destDomain` while the contract uses `srcDomain` and `dstDomain`.

```
// SDK sourceDomain, destDomain // Contract srcDomain,
dstDomain
```

Impact : Potential confusion for developers, but does not affect functionality if the mapping is correct.

Recommendation : Align nomenclature between SDK and contracts to avoid confusion.

MEDIUM #5 : No Limit on Number of Channels in AdaptiveRouter MEDIUM

 File: contracts/AdaptiveRouter.sol, Line 199

The loop in `routeStep()` iterates over all channels without limit, which can cause gas issues if too many channels are registered.

```
for (uint256 i = 0; i < len; i++) { // ⚠ No limit // ...
}
```

Impact : If a large number of channels are registered, `routeStep()` can exceed the gas limit and fail.

Recommendation : Add a maximum limit on the number of channels, or implement pagination for iterations.

LOW #1 : No Verification that amount > 0 Before Some Calculations

LOW

 Multiple contracts

Some contracts check `amount > 0` but not all consistently. For example, in `RecurPullSafeV2`, the check is done after other checks.

Impact : Low, but can cause unnecessary gas if `amount = 0` is passed.

Recommendation : Check `amount > 0` first in all functions that accept an amount.

LOW #2 : Missing Documentation on Gas Limits

LOW

 General documentation

The documentation does not explicitly mention gas limits or recommendations on the maximum number of channels/domains.

Impact : Low, but developers could deploy configurations that cause gas issues.

Recommendation : Add a section in the documentation on recommended limits and gas considerations.

Recommended Deployment Checklist

-  Verify that all controllers are contracts (Safe/multisig), not

EOAs

-  Ensure all trustedExecutors are trusted contracts
-  Test with non-standard tokens (USDT) to verify compatibility
-  Limit the number of channels/domains to avoid gas issues
-  Update the SDK to fix inconsistencies with contracts
-  Implement SafeERC20 before production deployment
-  Add reentrancy guards or explicitly document trust dependencies
-  Test race condition scenarios on ownerOfAuth
-  Verify that validAfter and metadataHash are correctly handled in the SDK

Conclusion

Global Verdict : The project presents a **solid and well-designed architecture** with appropriate security mechanisms for a non-custodial system. However, **several critical issues must be fixed before production deployment**, notably:

- The race condition on `ownerOfAuth` in `RecurConsentRegistry`
- The SDK/Contract inconsistency for `FlowIntent` (`validAfter`, `metadataHash`)
- The absence of `SafeERC20` for compatibility with all ERC-20 tokens
- Overflow risks in `FlowChannelHardened`

Recommendation : Fix critical and high issues before any production deployment. Medium issues can be addressed in a

later version but should be documented.

Report Generated On : November 10, 2025

Auditor : DEKOJO

Note : This report is based on static code analysis. A complete formal audit with tests and formal verification is recommended before production deployment.